LeetCode Java Coding Patterns & Tips

Detect Duplicates Using HashSet

```
Set<Integer> seen = new HashSet<>();
for (int num : nums) {
if (seen.contains(num)) {
return true;
}
seen.add(num);
}
return false;
```

Two Pointer Technique (Sorted Array)

```
int left = 0, right = arr.length - 1;
while (left < right) {
int sum = arr[left] + arr[right];
if (sum == target) return true;
if (sum < target) left++;
else right--;
}
return false;
```

Sliding Window (Longest Substring Without Repeating Characters)

```
Set<Character> set = new HashSet<>();
int left = 0, max = 0;
for (int right = 0; right < s.length(); right++) {
while (!set.add(s.charAt(right))) {
set.remove(s.charAt(left++));
}
max = Math.max(max, right - left + 1);
}
return max;
```

Binary Search

```
int left = 0, right = arr.length - 1;
while (left <= right) {
int mid = left + (right - left) / 2;
if (arr[mid] == target) return mid;
else if (arr[mid] < target) left = mid + 1;
else right = mid - 1;
}
return -1;
```

DP - Fibonacci (Bottom Up)

```
if (n <= 1) return n;
int[] dp = new int[n + 1];
dp[0] = 0; dp[1] = 1;
for (int i = 2; i <= n; i++) {
```

```
dp[i] = dp[i - 1] + dp[i - 2];
}
return dp[n];
```

Greedy - Jump Game

```
int reachable = 0;
for (int i = 0; i < nums.length; i++) {
if (i > reachable) return false;
reachable = Math.max(reachable, i + nums[i]);
}
return true;
```

DFS on Grid

```
void dfs(int[][] grid, int i, int j) {
if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length
|| grid[i][j] == 0) return;
grid[i][j] = 0; // mark visited
dfs(grid, i+1, j);
dfs(grid, i-1, j);
dfs(grid, i, j+1);
dfs(grid, i, j-1);
}
```

BFS (Shortest Path in Grid)

```
Queue<int[]> q = new LinkedList<>();
q.offer(new int[]{0, 0});
while (!q.isEmpty()) {
int[] cell = q.poll();
for (int[] dir : dirs) {
int r = cell[0] + dir[0], c = cell[1] + dir[1];
// check bounds and visited
}
}
```